



SLAM / Nav2 / MoveIt2 / Gazebo

▼ 하기 전 세팅 방법

💡 사용할 전체 환경 및 버전

구분	기준	구분	기준
Host OS	Windows 10	Language	Python 중심
Container OS	Ubuntu 22.04	Build Tool	colcon
ROS2	Humble Hawksbill	Container Tool	Docker Desktop + Docker Compose
Simulator	Gazebo Classic	GUI	VcXsrv
Navigation	Nav2	AI 개발도구	Google Antigravity
Mapping	SLAM Toolbox	소스 유지	Docker volume bind mount
Manipulation	MoveIt2	환경 유지	Dockerfile 기반 이미지 재빌드

📁 사전 폴더 생성

```

ros2-humble-lab/
├── config/           # ROS2 노드 및 파라미터 (.yaml 등)
├── maps/             # SLAM으로 생성한 지도 데이터 (.yaml, .pgm)
├── scripts/          # 파이썬 스크립트 또는 쉘 파일 (.py, .sh)
├── workspace/        # ROS2 워크스페이스 (src, build, install, log)
├── docker-compose.yml # 여러 도커 컨테이너를 정의하고 실행하는 설정 파일
└── Dockerfile        # 환경 설정을 위한 도커 이미지 설정 파일

```

▼ Dockerfile 코드 (ROS2, Gazebo Classic, SLAM, Nav2, TurtleBot3, MoveIt2, 등 패키지 설치)

```

FROM ubuntu:22.04

ENV DEBIAN_FRONTEND=noninteractive
ENV ROS_DISTRO=humble
ENV LANG=en_US.UTF-8
ENV LC_ALL=en_US.UTF-8

# -----
# 1. 기본 패키지 설치
# -----

RUN apt-get update && apt-get install -y \
    locales \
    curl \
    wget \
    gnupg2 \
    lsb-release \
    software-properties-common \
    ca-certificates \
    build-essential \
    git \
    vim \

```

```

nano \
gedit \
terminator \
net-tools \
iputils-ping \
python3 \
python3-pip \
python3-venv \
python3-dev \
&& locale-gen en_US en_US.UTF-8 \
&& update-locale LC_ALL=en_US.UTF-8 LANG=en_US.UTF-8 \
&& rm -rf /var/lib/apt/lists/*

# -----
# 2. ROS2 apt 저장소 추가
# -----
RUN add-apt-repository universe

RUN curl -sSL https://raw.githubusercontent.com/ros/rosdistro/master/ros.key \
-o /usr/share/keyrings/ros-archive-keyring.gpg

RUN echo "deb [arch=$(dpkg --print-architecture) signed-by=/usr/share/keyrings/r
os-archive-keyring.gpg] http://packages.ros.org/ros2/ubuntu $(. /etc/os-release
&& echo $UBUNTU_CODENAME) main" \
> /etc/apt/sources.list.d/ros2.list

# -----
# 3. ROS2 Humble Desktop 설치
# -----
RUN apt-get update && apt-get install -y \
ros-humble-desktop \
ros-dev-tools \
python3-colcon-common-extensions \
python3-rosdep \
python3-vcstool \
&& rm -rf /var/lib/apt/lists/*

# -----
# 4. Gazebo Classic 및 ROS2 연동 패키지 설치
# -----
RUN apt-get update && apt-get install -y \
gazebo \
ros-humble-gazebo-ros-pkgs \
ros-humble-gazebo-ros2-control \
&& rm -rf /var/lib/apt/lists/*

# -----
# 5. SLAM / Nav2 / TurtleBot3 패키지 설치
# -----
RUN apt-get update && apt-get install -y \
ros-humble-slam-toolbox \
ros-humble-navigation2 \
ros-humble-nav2-bringup \
ros-humble-turtlebot3 \
ros-humble-turtlebot3-msgs \

```

```

ros-humble-turtlebot3-simulations \
&& rm -rf /var/lib/apt/lists/*

# -----
# 6. MoveIt2 설치
# -----
RUN apt-get update && apt-get install -y \
    ros-humble-moveit \
    ros-humble-moveit-setup-assistant \
    ros-humble-ros2-control \
    ros-humble-ros2-controllers \
    && rm -rf /var/lib/apt/lists/*

# -----
# 7. Python 개발 편의 패키지 설치
# -----
RUN pip3 install --no-cache-dir \
    numpy \
    scipy \
    matplotlib \
    pandas \
    opencv-python

# -----
# 8. rosdep 초기화
# -----
RUN rosdep init || true
RUN rosdep update || true

# -----
# 9. ROS2 환경변수 자동 설정
# -----
RUN echo "source /opt/ros/humble/setup.bash" >> /root/.bashrc
RUN echo "export TURTLEBOT3_MODEL=waffle" >> /root/.bashrc
RUN echo "export GAZEBO_MODEL_PATH=\$GAZEBO_MODEL_PATH:/opt/ros/humble/share/turtlebot3_gazebo/models" >> /root/.bashrc

# -----
# 10. 작업 디렉터리 생성
# -----
RUN mkdir -p /root/ros2_ws/src
WORKDIR /root/ros2_ws

# -----
# 11. 기본 실행 셸
# -----
CMD ["/bin/bash"]

```

▼ docker-compose.yml 코드

```

services:
  ros2-humble:
    build:
      context: .
      dockerfile: Dockerfile

```

```

image: ros2-humble-lab:latest
container_name: ros2-humble-lab

stdin_open: true
tty: true

environment:
  - DISPLAY=host.docker.internal:0.0
  - QT_X11_NO_MITSHM=1
  - LIBGL_ALWAYS_SOFTWARE=1
  - ROS_DOMAIN_ID=30
  - TURTLEBOT3_MODEL=waffle

volumes:
  - ./workspace:/root/ros2_ws
  - ./config:/root/config
  - ./maps:/root/maps
  - ./scripts:/root/scripts

working_dir: /root/ros2_ws

network_mode: bridge

shm_size: "2gb"

command: /bin/bash

```

이후 docker-compose build / up -d / exec 로 들어가면 된다.

현재 위치 확인하고, 아래 기본 패키지를 설치해주면 된다.

```

mkdir src && cd src

ros2 pkg create py_basic_pubsub \
  --build-type ament_python \
  --dependencies rclpy std_msgs

```

전체적인 tree는 아래와 같다.

```

ros2_ws
├── src
│   ├── py_basic_pubsub
│   │   ├── package.xml
│   │   ├── py_basic_pubsub
│   │   │   ├── __init__.py
│   │   │   └── simple_publisher.py    ← 이걸 이제 만들어줘야 함.
│   │   ├── resource
│   │   │   └── py_basic_pubsub
│   │   ├── setup.cfg
│   │   ├── setup.py
│   │   └── test
│   │       ├── test_copyright.py
│   │       └── test_flake8.py

```

```
|   └─ test_pep257.py
└─ test.txt
```

▼ simple_publisher.py 코드 (예시 Node 파일)

```
import rclpy
from rclpy.node import Node
from std_msgs.msg import String

class SimplePublisher(Node):
    def __init__(self):
        super().__init__('simple_publisher')

        self.publisher_ = self.create_publisher(
            String,
            'robot_status',
            10
        )

        self.count = 0

        self.timer = self.create_timer(
            1.0,
            self.publish_status
        )

        self.get_logger().info('Simple Publisher node has started.')

    def publish_status(self):
        msg = String()
        msg.data = f'Robot is running. Count: {self.count}'

        self.publisher_.publish(msg)

        self.get_logger().info(f'Publishing: {msg.data}')

        self.count += 1

def main(args=None):
    rclpy.init(args=args)

    node = SimplePublisher()

    try:
        rclpy.spin(node)
    except KeyboardInterrupt:
        node.get_logger().info('Publisher node stopped by user.')
    finally:
        node.destroy_node()
        rclpy.shutdown()
```

```
if __name__ == '__main__':
    main()
```

▼ simple_subscriber.py 코드 (예시 Node 파일)

```
import rclpy
from rclpy.node import Node
from std_msgs.msg import String

class SimpleSubscriber(Node):
    def __init__(self):
        super().__init__('simple_subscriber')

        self.subscription = self.create_subscription(
            String,
            'robot_status',
            self.listener_callback,
            10
        )

        self.get_logger().info('Simple Subscriber node has started.')

    def listener_callback(self, msg):
        self.get_logger().info(f'I heard: {msg.data}')

def main(args=None):
    rclpy.init(args=args)

    node = SimpleSubscriber()

    try:
        rclpy.spin(node)
    except KeyboardInterrupt:
        node.get_logger().info('Subscriber node stopped by user.')
    finally:
        node.destroy_node()
        rclpy.shutdown()

if __name__ == '__main__':
    main()
```

Node를 만들었다면 `setup.py` 에 등록해줘야 함. (맨 마지막 entry_points에 추가해줘야 함.)

`ros2 run [패키지명] [별명]` 쳤을 때, 실행될 코드가 어디 있는지 알려주는 '바로가기 버튼'을 만드는 작업

```
entry_points={
    'console_scripts': [
        'simple_publisher = py_basic_pubsub.simple_publisher:main',
        'simple_subscriber = py_basic_pubsub.simple_subscriber:main',
    ],
},
```

다 했다면, src 바로 위 폴더로 가서 `colcon build` 해줘야 함. 반드시 "src폴더 밖에서"

`install` 폴더에 실행에 필요한 모든 파일이 정돈시키는 작업

```
colcon build
```

이후에 아래걸로 해줘야 함.

터미널 창은 콜콘 해줘도 모름. '새로고침 버튼 누르기' 작업

```
source install/setup.bash
```

실행 방식

```
# 터미널 창 1
ros2 run py_basic_pubsub simple_publisher # .py는 생략하고 해야 함.

# 터미널 창 2
ros2 run py_basic_pubsub simple_subscriber # .py는 생략하고 해야 함.
```

▼ SLAM

Simultaneous Localization and Mapping의 약자.

쉽게 말하면 로봇이 다음 두 가지를 동시에 하는 기술.

1. 내가 어디 있는지 파악한다.
2. 주변 지도를 만든다.

ex. 로봇청소기

▼ Nav2

로봇의 이동경로를 계획하는 ROS2 대표 패키지.

하는 일은 다음과 같다.

1. 현재 위치 파악
2. 목표 지점 설정
3. 경로 계획
4. 장애물 회피
5. 속도 명령 생성
6. 목표 지점 도착 판단

쉽게 얘기하면, 이동형 로봇의 주행을 담당. (로봇의 다리)

▼ MoveIt2

로봇팔 제어와 경로 계획을 위한 ROS2 대표 패키지.

하는 일은 다음과 같다.

- 로봇팔 관절 제어
- 목표 자세 계산
- 충돌 없는 경로 계획
- 그리퍼 제어
- Pick & Place 작업

쉽게 얘기하면, 로봇팔의 움직임을 담당. (로봇의 팔)

▼ Gazebo (Classic)

실제 장비 없이 가상 환경에서 테스트하는 시뮬레이터

실제 Gazebo로 하는 일은 아래와 같다.

- 실제 로봇 투입 전 검증
- 센서 데이터 테스트
- 자율주행 알고리즘 검증
- 로봇팔 동작 시뮬레이션